

Introduction to Generative AI

AI Bootcamp · Kalvium

Session Overview

This session introduces Generative AI from the beginning. Students do not need prior AI experience.

By the end of this session, students will understand what Generative AI is, how developers use it through APIs, and how to run their first Gemini API call in Google Colab.

Learning Outcomes

After this session, students will be able to:

- Explain Generative AI in simple words.
- Distinguish between AI, Machine Learning, Deep Learning, and Generative AI.
- Explain the difference between traditional ML and API-driven AI development.
- Understand basic LLM concepts: prompt, token, context window, temperature, max output tokens.
- Make a first Gemini API call using Python.
- Experiment with temperature and max output token settings.
- Understand why API cost and token usage matter.

1. What Is Artificial Intelligence?

Artificial Intelligence (AI) is a broad field of computer science where we build systems that can perform tasks normally requiring human intelligence.

Examples of AI tasks:

- Recognizing speech
- Understanding and generating text
- Translating languages
- Detecting objects in images
- Recommending videos or products

In simple words: AI is any technique that enables computers to mimic human intelligence, logic or decision-making.

Ex. A traditional chess engine that evaluate millions of board positions using a hard coded scoring system to find the best move.

2. AI → ML → Deep Learning → Generative AI

These terms are related but distinct. Think of them as nested circles:

- **AI** — The broad field of making machines intelligent.
- **Machine Learning** — A part of AI where systems *learn patterns from data* instead of being manually programmed.
- **Deep Learning** — A part of ML that uses neural networks with many layers.
- **Generative AI** — A part of AI that *creates new content* (text, code, images, audio, video).

ML Example — Will a Student Get Placement?

CGPA	Projects	Placement
6.5	1	No
8.0	3	Yes
7.5	2	Yes
6.0	1	No
7.8 (new)	2	Model predicts: Yes

Pattern: *Past Data* → *Learn Pattern* → *Predict Future Outcome*

3. What Is Generative AI?

Generative AI is a type of AI that **creates new content** based on the input it receives.

It can generate: Text · Code · Images · Study plans · Summaries · Chat responses · Ideas

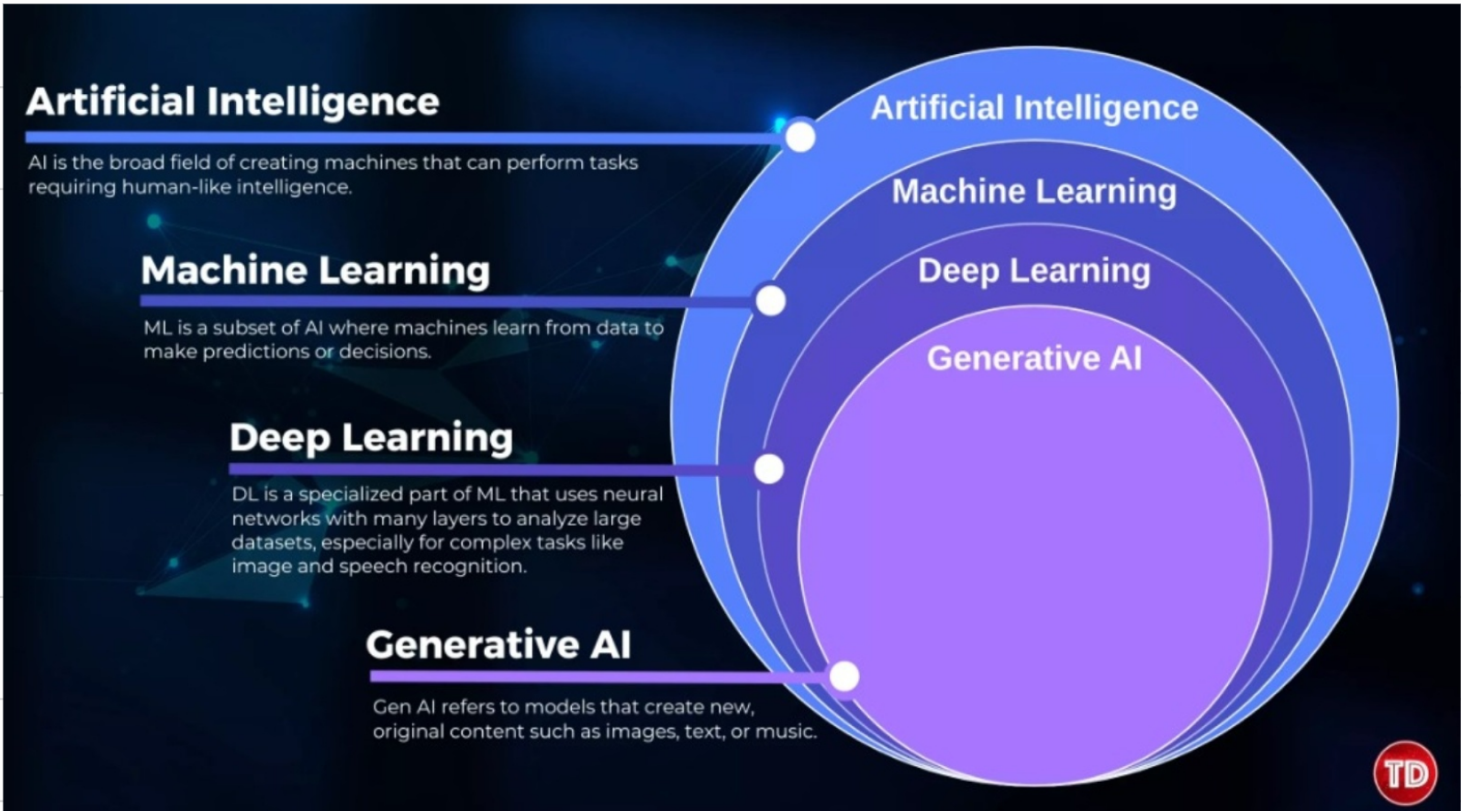
Example Prompt:

Explain photosynthesis to a 10-year-old in 5 simple lines.

Possible Model Response:

Photosynthesis is how plants make their food.
Plants use sunlight, water, and carbon dioxide.
Their leaves collect sunlight.
They make sugar for energy.
They also release oxygen into the air.

■ **Important:** *The model did not copy a fixed answer. It generated a new response based on the prompt and its learned patterns.*



5. What Is an LLM?

LLM = Large Language Model — A model trained on a very large amount of text. It learns patterns in language and can generate human-like responses.

Examples: ChatGPT · Gemini · Claude · GitHub Copilot

What LLMs can do:

- Answer questions & explain difficult topics
- Write, debug, and summarize code
- Summarize long text and rewrite content
- Translate text and brainstorm ideas

■ ■ **Important limitation:** LLMs can make mistakes. They can sound confident even when the answer is wrong. Developers must verify outputs carefully.

6. How Does an LLM Work?

ChatGPT is based on **GPT: Generative Pre-trained Transformer**

- **Generative** → generates new text (answers, code, explanations)
- **Pre-trained** → trained on huge data before being used
- **Transformer** → the neural network architecture that makes it powerful

Core idea: Given some text, predict the most likely next word.

Input: "Machine learning is"
Prediction: "powerful"

TRANSFORMER

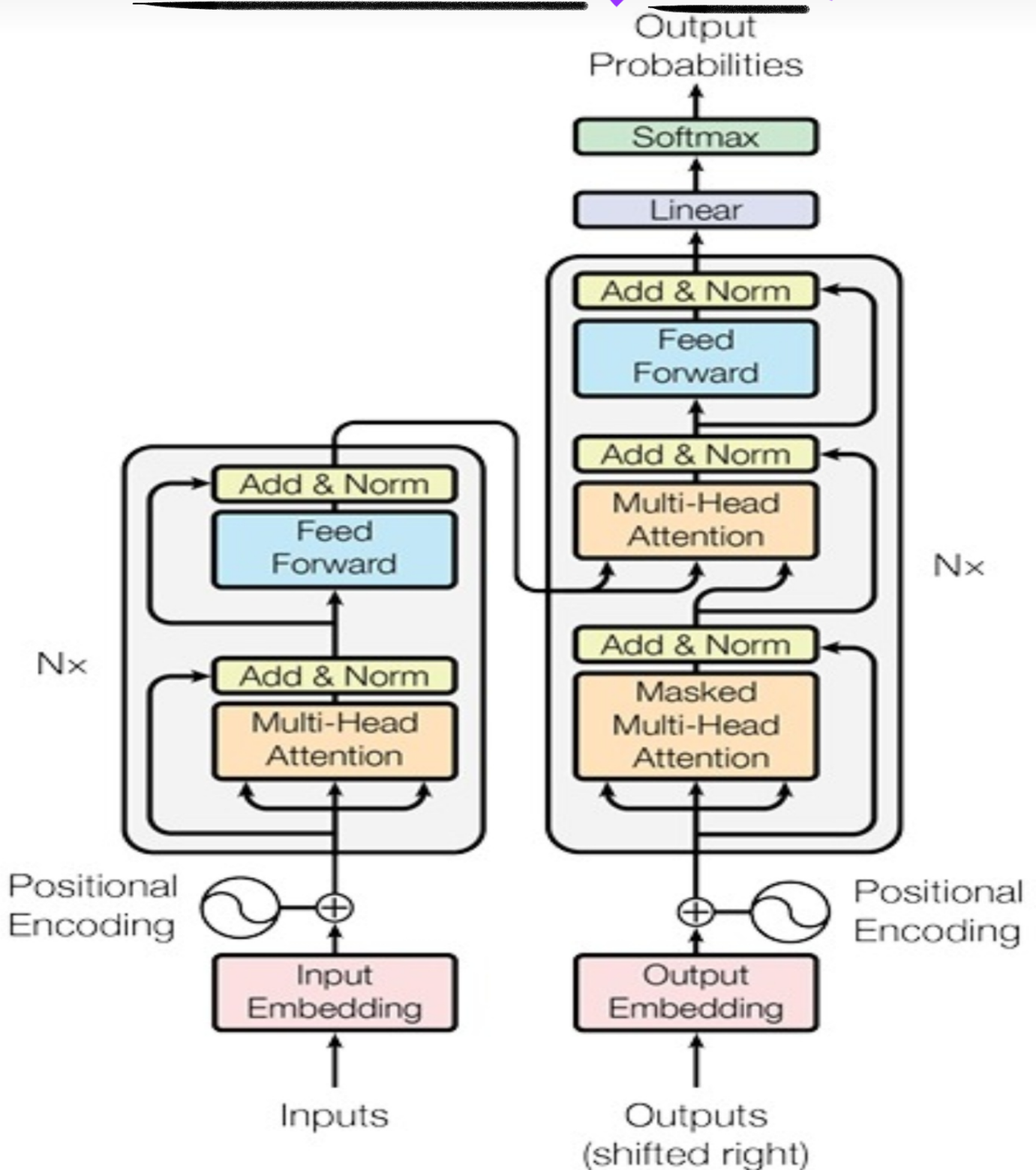


Figure 1: The Transformer - model architecture.

by Google (used in Translator)



Google translator.

In case of GPT or other LLM Transformer predict the next word.

ex.



- Computer does not understand the english language.

Step 1: Encoding (Input embedding)

Step 1 — Tokenization

Computers do not understand raw text. The input sentence is broken into **tokens** (small pieces of words).

→ Sentence: "Machine learning is amazing"
Tokens: ["Machine", "learning", "is", "amazing"]

unbelievable → ["un", "believ", "able"]

- Vary model to model.

- dictionary will map these tokens with some numbers.

[_ , _ , _ , _]

Step 2 — Vector Embeddings

Every token is converted into a **vector embedding** (a list of numbers capturing meaning).

"king" → [0.12, 0.91, 0.33, ...]

"queen" → [0.10, 0.89, 0.35, ...]

Key relationship: king - man + woman ≈ queen

milk ←→ cow
Human ← cat

Vector embeddings are numerical representations of data (text, images, audio) that capture semantic meaning and relationships as lists of numbers (vectors) in a multidimensional space.

They enable AI models to understand context, where similar data points are clustered closely together. Common synonyms include numerical representations, embedding vectors, data vectors, or simply embeddings.  Elastic +4

- Capturing the real meaning of a word.

Step 3 — Positional Encoding

Transformers process tokens in parallel. Positional encoding tells the model the order of words.

"Dog bites man" vs "Man bites dog"

→ Same words, different meanings → positional encoding resolves this.

It add the position of tokens with its vector embedding matrix.

Step 4 — Transformer Architecture (Self-Attention)

Self-attention allows each word to look at other words to understand context.

"The animal didn't cross the street because it was tired."

→ "it" refers to "animal" - self-attention learns this link.

Each word asks:

Query → What am I looking for?

Key → What information do I contain?

Value → What should I pass forward?

Multi-Head Attention: The model looks at many relationships simultaneously (grammar, subject-verb, meaning, long-distance dependencies).

Step 5 — Next Token Prediction

Input: "Artificial intelligence will"
Predictions: change→0.40 transform→0.30 improve→0.15 be→0.10
→ Model selects the most likely word and continues.

Step 6 — Pre-Training

LLMs are trained on massive datasets (books, websites, research papers, code). The model repeatedly predicts hidden words and adjusts its weights.

Input: "Deep learning models require large ____"
Answer: "datasets"

Final Pipeline:

User Text → Tokenization → Vector Embeddings → Positional Encoding
→ Transformer Layers (Self-Attention + Neural Networks)
→ Next Token Prediction → Generated Response

■ **Key idea:** *An LLM does not store answers like a database. It learns patterns and uses them to generate text intelligently.*

7. Traditional Machine Learning Workflow

Traditional ML typically follows this pipeline:

Collect Data → Clean Data → Train Model → Test Model → Deploy Model → Monitor Model

Example: Build a spam email detector.

- Collect many emails & label each as spam / not spam
- Clean the data, train a model, test it
- Deploy and monitor whether it keeps working

■ *This approach is powerful, but requires data, labels, training, testing, and deployment — it takes time.*

8. API-Driven Generative AI Workflow

With Generative AI APIs, developers use an **already-trained model** through an API.

User Input + Prompt → LLM API → Model Response → App Logic → User

Example: Build a tool that explains whether an email looks suspicious.

- Take email text from the user
- Create a prompt asking the model to classify and explain
- Send the prompt to the LLM API & receive the response
- Show the result in the app

Terminology :-

Context Window

This is the model's "short-term memory." It is the maximum number of tokens the model can process at one time (including both your prompt and its response).

- **Practical Example:** If an LLM has a 8k token context window, you cannot paste a 20,000-line Java codebase into it and ask it to find a bug. It will "forget" the beginning of the code before it reaches the end.

11. What Is Temperature?

Temperature controls how **random or creative** the response is.

Temperature	Behavior	Best For
Low (0.1–0.3)	Predictable, consistent	Classification, factual Q&A, data extraction
Medium (0.5–0.7)	Balanced	General explanations, summaries
High (0.9–1.0)	Creative, varied	Brainstorming, taglines, story ideas

Example — Same prompt at different temperatures:

Prompt: "Write a slogan for a plant store."

Temp 0.2: "Grow green, live fresh."

Temp 1.0: "Where tiny leaves become wild little forests."